An Adaptive Game-Theory Defense Framework Against Markovian Stealth Attacks in Software-Defined Networks

Prepared by

Shouq Alkhashman

Saud Al-Haji

Supervised by

Dr. Wassim Juneidi

Eng. Mohammad Yamen Hallak

2026-2027

Abstract

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
SDN	Software-Defined Networking
DDoS	Distributed Denial-of-Service
IDS	Intrusion Detection System
OVS	Open vSwitch
RTT	Round-Trip Time
MDP	Markov Decision Process
ICMP	Internet Control Message Protocol
QoS	Quality of Service

List of Symbols

Symbol	Meaning
P(i, j)	Transition probability from state i to state j
π	Stationary distribution of the Markov chain
β	Discount factor for future rewards ($0 \leq \beta \leq 1$)
α	Malicious-reputation update coefficient
V(s)	Value of state s in the stochastic game
R(s, a)	Immediate reward of action a in state s

Introduction

1.1 General and Specific Significance of the Field

Network security has become one of the most critical pillars of modern information technology. As organizations increasingly rely on interconnected digital infrastructures, the need to protect networks against malicious activities, service disruption, and unauthorized access has grown accordingly. Among the many paradigms that have reshaped networking in the last decade, Software-Defined Networking (SDN) stands out for separating the control plane from the data plane and concentrating network intelligence in a programmable controller. This architectural shift has improved manageability, scalability, and flexibility, making SDN a foundation for cloud computing, network virtualization, and security-aware infrastructures.

However, the same centralization that gives SDN its power also introduces a critical vulnerability. The controller, being the logical core of the network, becomes a strategic target for adversaries. Protecting the SDN controller against resource-exhaustion and flooding-based attacks is therefore a central concern, and it motivates the specific focus of this project on adaptive, intelligent defense mechanisms.

1.2 Problem Statement

Although SDN provides centralized control and fine-grained programmability, it creates a critical security challenge: the controller may act as a single point of failure. When it is targeted by DDoS attacks or overwhelmed by abnormal flow requests, the entire network may suffer from delay, packet loss, and degraded quality of service. The problem becomes even harder when the attacker behaves intelligently and adaptively, deliberately keeping malicious traffic below conventional detection thresholds so that signature-based or fixed-threshold defenses fail to react in time.

Consequently, the core problem addressed by this project is the absence of a practical, adaptive, and lightweight security framework for SDN that can detect intelligent, time-varying attack behavior and respond with an appropriate, strategically chosen mitigation action, without imposing excessive overhead on the controller.

1.3 Project Objective

The reference study on which this project builds, SEC-SDN (Razvan and Mitica, 2025), proposed a security-enhanced SDN framework that integrates the Snort intrusion detection system with game-theoretic modeling to improve resilience against DDoS attacks. It

demonstrated that combining centralized SDN control, intrusion detection, and strategic defense logic within a single framework is both feasible and effective.

Building on this foundation, the aim of this project is to design and implement an adaptive defense framework that goes a step further by modeling the intelligent attacker as a Markov chain that transitions among multiple attack states, and by extending the decision engine from static games to stochastic (Markov) games. The goal is to detect complex, stealthy attacks and to select the optimal mitigation response automatically over time, while studying the impact of these decisions on network performance.

1.4 Related Work

This section reviews ten peer-reviewed studies published in 2025 and 2026, excluding the SEC-SDN reference study on which the project is based. The selected works span game-theoretic and reinforcement-learning-based adaptive defense, stealthy and low-rate attack detection, SDN-specific attacks such as flow-table exhaustion and control-plane saturation, and practical detection and mitigation frameworks validated on Mininet-based testbeds.

[1] Shao et al. (2025) proposed AdaDoS, an adaptive DoS attack framework that models the attacker and the detector as rivals in a competitive game, letting the attacker adapt its traffic from SDN feedback to evade both machine-learning and rule-based detectors, and framing the problem as a partially observable decision process. This is the most directly relevant study to the project, since it formalizes the adaptive, game-based attacker that the project's Markov attacker and stochastic-game defender are designed to counter.

[2] Kachavimath and Narayan (2025) proposed a hybrid approach combining multi-feature selection, unsupervised clustering, and game theory to detect and mitigate DDoS attacks in SDN, scoring suspicious hosts by reputation and using a game-theoretic payoff model to guide adaptive mitigation. It is highly relevant because it links detection to game-theoretic mitigation, which is the core idea of this project.

[3] McGugan, Cook, and Pezaros (2026) introduced a multi-agent reinforcement-learning framework built on SDN that mitigates denial-of-service attacks in near real time, using network-state measurements within a hierarchical action space and recovering from attacks faster than a single-agent approach. It provides a methodology for distributed, multi-agent defense that could extend the framework to several controllers or several simultaneous attackers.

[4] You et al. (2026) proposed DPAD, a dual-perspective anomaly detector that combines per-flow statistical deviation with host-level behavioral entropy, flagging a flow only when a strong deviation and a sharp entropy drop occur together. This is highly relevant because the entropy indicator is an excellent additional signal for detecting the slow stealth state (S1) and distributed spoofing in the proposed Markov attacker model.

[5] Swileh and Zhang (2025) presented a framework for decentralized SDN that collects lightweight port-level counters and classifies them using a pre-trained large language model without fine-tuning, then drops traffic at the attacker's port until the attack stops. It is an example of lightweight, controller-side detection based on simple counters, supporting the project's emphasis on low operational overhead near the controller.

[6] Tang et al. (2026) modeled low-rate flow-table overflow attacks, in which an adversary gradually installs malicious rules until the switch's limited flow table is full and legitimate traffic is blocked, and proposed an online system that ranks rules by eviction priority with about 95% accuracy. This is the key reference for the project's planned extension of the threat model to flow-table exhaustion.

[7] Ji et al. (2025) proposed a detector for saturation attacks in SDN using Multi-edge Communication Flow Graphs and an edge representation-learning model with attention, reporting high accuracy on two topologies. It is relevant because it covers both flow-table and controller saturation within a single framework, matching the project's expanded threat vectors.

[8] Irtaish and Hamarsheh (2025) presented a live defense system for SDN built on the ONOS controller, combining machine learning with efficient Python tools to detect and stop UDP and TCP SYN flood attacks, reporting 98.5% accuracy and a 3.2-second response time on a Mininet topology. It is a practical, end-to-end detection-and-mitigation pipeline similar to the project's own setup.

[9] Mahar et al. (2026) built a dataset from a Mininet testbed running a Ryu controller and Open vSwitch, deriving SDN-specific flow features and combining Random Forest and XGBoost into a hybrid classifier with about 99.36% accuracy, though it evaluates detection only. It is directly relevant because it uses the same Mininet/Ryu/Open vSwitch environment as the project.

[10] Labesh Kumar et al. (2025) proposed a metaparameter-optimized hybrid deep-learning model for SDN security, using a binary Narwhal Optimizer for feature selection and a

CNN-BiGRU model with attention tuned by the Seagull Optimization Algorithm, reaching about 99.4% accuracy. It serves as a high-accuracy detection baseline against which the project's lightweight approach can be compared.

Table 1.1 Summary of Related Work .

#	Author(s)	Venue	Year	Relation to the Project
1	Shao et al.	arXiv	2025	Adaptive game-based attacker (closest to the project)
2	Kachavimath, Narayan	J. Telecom. & Info. Technology	2025	Detection linked to game-theoretic mitigation
3	McGugan et al.	Computer Networks (Elsevier)	2026	Scalable distributed multi-agent defense
4	You et al.	EURASIP J. Wireless Comm. (Springer)	2026	Entropy signal for stealth/spoofing detection
5	Swileh, Zhang	arXiv	2025	Lightweight controller-side detection
6	Tang et al.	IEEE Trans. Computers	2026	Flow-table exhaustion modeling
7	Ji et al.	J. King Saud Univ. – CIS	2025	Flow-table and controller saturation
8	Irtaish, Hamarsheh	ENG4PAL Conf. (PPU)	2025	End-to-end detection+mitigation on Mininet
9	Mahar et al.	Scientific Reports (Nature)	2026	Same Mininet/Ryu/OVS environment
10	Labesh Kumar et al.	Scientific Reports (Nature)	2025	High-accuracy detection baseline

1.5 Practical Applications of the Project

The proposed framework has practical relevance in both operational and academic contexts. In enterprise and cloud SDN environments, an adaptive framework that links traffic inspection to controller-based response can improve resilience against DDoS attacks by reducing response time and helping preserve service continuity under adversarial conditions. Because it can be developed in a simulated SDN environment using tools such as Mininet, Ryu, and Snort, it also provides a practical platform for studying SDN security, IDS integration, and attacker-defender modeling.

1.6 Scientific Challenges

The project is associated with several technical and research challenges. First, processing time is critical: any delay introduced by inspection, analysis, or mitigation logic may reduce the effectiveness of the defense. Second, false alarms remain a concern, as detection systems may classify legitimate traffic as malicious, leading to unnecessary blocking and degraded quality of

service. Third, resource overhead must be carefully controlled, so that controller-side security does not consume excessive CPU, memory, or bandwidth.

Finally, the adaptive strategy selection itself is demanding. Detecting suspicious traffic is only the first step; the framework must still decide how to respond, and this decision cannot rely on fixed thresholds alone. Modeling the attacker as a Markov process and computing a dynamic equilibrium in a stochastic game is intellectually and computationally challenging, which is precisely why these techniques form the core research contribution of this project.

Chapter One: Theoretical Foundations

2.1 Introduction

This chapter presents the theoretical foundations on which the project is built, focusing on the concepts, technologies, and security principles directly used in the design of the proposed adaptive framework. It begins with Software-Defined Networking and its layered architecture, then describes the OpenFlow protocol, the Ryu controller, and traffic mirroring in Open vSwitch. It continues with intrusion detection systems and the Snort engine, the characteristics of DDoS attacks and IP spoofing in SDN, and the advanced SDN-specific attacks targeted for extension. Finally, it introduces game theory, Markov chains, and stochastic games, which together form the mathematical basis for modeling an intelligent, adaptive attacker and for computing an optimal, time-aware defensive policy.

2.2 Software-Defined Networking (SDN)

Software-Defined Networking is a modern networking paradigm that separates the control plane, which decides how traffic is handled, from the data plane, which actually forwards the traffic. In traditional networks, these two planes are tightly coupled inside each device, which makes network-wide configuration slow, error-prone, and difficult to adapt. SDN moves control intelligence into a centralized, programmable controller that maintains a global view of the entire network and installs forwarding rules in simple, programmable switches.

2.2.1 Comparison with Traditional Networks

The most important distinction for this project is programmability: because the SDN controller can install and remove rules dynamically, it can react to detected threats in near real time, which is the basis of adaptive defense. Table 2.1 summarizes the main differences.

Table 2.1 Traditional networking versus Software-Defined Networking.

Dimension	Traditional Networking	Software-Defined Networking
Control logic	Distributed inside each device	Centralized in a programmable controller
Configuration	Device-by-device, manual	Centralized and automated
Global view	None; each device sees locally	Complete, maintained by the controller
Programmability	Limited and vendor-specific	High, through open APIs
Security response	Mostly manual	Can be automated at the controller

2.2.2 Significance for the Project

Because the controller maintains a global view and can update forwarding behavior dynamically, it provides the technical basis for moving from passive monitoring to active, programmable mitigation. At the same time, this centralization creates a strategic point of

exposure, which is precisely why protecting the controller is the central security concern of this work.

2.3 SDN Layered Architecture

The SDN architecture is commonly described as a three-layer model consisting of the application layer, the control layer, and the infrastructure layer, connected by two programmable interfaces.

2.3.1 The Three Layers

The application layer hosts network services such as monitoring, policy management, and security applications. The control layer contains the SDN controller, which acts as the logical decision-making core, translates high-level intent into forwarding rules, and maintains the global network view. The infrastructure layer, also called the data plane, consists of the forwarding devices such as OpenFlow switches, which forward packets according to the rules installed by the controller and report statistics and table-miss events back to it.

2.3.2 Northbound and Southbound Interfaces

The northbound interface connects the controller to the applications above it, allowing high-level policies to be expressed, while the southbound interface connects the controller to the switches below it, most commonly using OpenFlow. This separation enables a structured security workflow in which suspicious traffic is observed at the data plane, evaluated at the control and application layers, and mitigated through controller-enforced actions.

2.4 The OpenFlow Protocol

OpenFlow is one of the most important southbound protocols in SDN, because it standardizes how the controller communicates with forwarding devices, allowing it to install, modify, and remove flow rules dynamically.

2.4.1 Flow Tables and Flow Entries

Each OpenFlow switch maintains one or more flow tables. Every flow entry consists of match fields that identify which packets it applies to, counters that record statistics, and actions applied to matching packets. When a packet arrives, the switch searches for a matching entry; if none is found, the packet is treated as a table miss. Table 2.2 lists representative match fields and actions.

Table 2.2 Representative OpenFlow match fields and actions.

Category	Examples
Match fields	Ingress port, source/destination MAC, source/destination IP, protocol, ports
Actions	Forward to port, drop, send to controller, modify header, apply meter (rate limit)
Counters	Packet count, byte count, flow duration

2.4.2 Packet-In, Packet-Out, and Meters

When a packet does not match any rule, the switch sends it to the controller as a Packet-In message; the controller can also inject packets using Packet-Out messages. This mechanism is directly relevant to the project, because an attacker can deliberately generate table-miss traffic to flood the controller with Packet-In messages. OpenFlow meters measure and limit the rate of traffic associated with a flow, and are the mechanism used to enforce the graduated rate-limiting decisions (RL_1, RL_2, RL_3), while severe attacks are handled by high-priority drop rules.

2.5 Ryu Controller and Traffic Mirroring in Open vSwitch

2.5.1 The Ryu Controller

In this project, the selected controller is Ryu, an open-source, component-based controller written in Python and widely used in academic environments. Ryu follows an event-driven model: it receives events from the switches, such as Packet-In messages and flow statistics replies, and applications respond by installing or modifying flow rules. This makes Ryu a convenient platform for implementing the proposed decision engine.

2.5.2 Traffic Mirroring in Open vSwitch

Open vSwitch supports traffic mirroring, which creates a copy of selected traffic and forwards it to a monitoring interface without altering the original forwarding path. In this project, each switch sends a copy of the traffic it observes to a dedicated Snort virtual machine. This separates forwarding from inspection and improves realism, because the IDS receives mirrored traffic like a monitoring sensor in real networks.

2.6 Intrusion Detection Systems and the Snort Engine

2.6.1 Intrusion Detection Systems

An Intrusion Detection System analyzes traffic or system behavior to identify malicious activity. Signature-based systems compare traffic against known attack patterns; anomaly-based systems flag deviations from a model of normal behavior; hybrid systems combine both. In SDN,

an IDS is especially valuable because detection can be linked directly to control logic, triggering enforcement actions rather than merely raising an alert.

2.6.2 The Snort Engine

Snort is a widely used, open-source IDS that performs rule-based inspection and generates alerts when suspicious signatures or patterns are detected. In the proposed framework, Snort represents the inspection layer that examines the mirrored traffic and provides the evidence needed for a controller-side decision, forming part of a wider workflow that links inspection to adaptive mitigation.

2.7 DDoS Attacks and IP Spoofing in SDN

2.7.1 Denial-of-Service and Distributed Denial-of-Service

A Denial-of-Service attack aims to make a service unavailable to legitimate users; a DDoS attack achieves this using many sources at once. DDoS attacks can be volumetric, protocol-based, or application-layer, as summarized in Table 2.3.

Table 2.3 Broad categories of DDoS attacks.

Category	Target	Examples
Volumetric	Network bandwidth	ICMP flood, UDP flood
Protocol	Connection/processing resources	TCP SYN flood, Packet-In storm
Application-layer	Specific services	HTTP flood, slow-rate attacks

2.7.2 DDoS in SDN and the Role of the Controller

In SDN, DDoS attacks are particularly dangerous because the centralized controller may be affected directly or indirectly through malicious flow activity. Excessive flow requests or Packet-In storms can overload the control channel, leading to delay, packet loss, and degraded quality of service, so protecting the controller is the central concern of this project.

2.7.3 IP Spoofing

An attacker may forge the source IP address to hide its identity or bypass source-based filtering. In SDN, spoofing can be detected by comparing the observed source address with the expected host identity or arrival port. When combined with ICMP flooding, the attack is both high-volume and identity-manipulated. Flooding and spoofing form the primary threat model used in the experimental part of this project.

2.8 Advanced SDN-Specific Attacks

Beyond conventional flooding and spoofing, this project considers attacks that specifically exploit the SDN architecture and are not visible in mirrored data traffic, requiring controller-side indicators.

2.8.1 Flow-Table Exhaustion

The flow-table exhaustion attack targets the limited capacity of a switch's flow table: an adversary gradually installs many malicious or short-lived rules until legitimate flows can no longer be accommodated. Even a low-rate attacker can degrade the switch over time while staying below volume-based thresholds, so detection requires monitoring the number of active flows per switch.

2.8.2 Control-Plane Saturation (Packet-In Storms)

The control-plane saturation attack exploits the Packet-In mechanism: crafted traffic with randomized headers continuously produces table misses, forcing the switch to flood the controller with Packet-In messages and saturating the control channel. Detection requires monitoring the Packet-In message rate. Both attacks are targeted for extension, and application-layer attacks may optionally be considered.

2.9 Game Theory in Cybersecurity

2.9.1 Fundamental Concepts

Game theory analyzes interactions among rational entities whose decisions influence one another. A game is defined by players, their strategies, and a payoff function. A central solution concept is the Nash equilibrium, in which no player can improve its payoff by unilaterally changing its strategy.

2.9.2 Types of Games

Two distinctions are especially important for this project: static versus dynamic/repeated games, since the adaptive attacker requires a repeated model; and complete versus partial information, since the defender only estimates the attacker's state. Table 2.4 summarizes these classifications.

Table 2.4 Classification of games relevant to the project.

Dimension	Types	Relevance to the project
Timing	Static vs. dynamic/repeated	Adaptive attacker needs a repeated model
Cooperation	Cooperative vs. non-cooperative	Attacker-defender is non-cooperative
Payoff sum	Zero-sum vs. general-sum	Zero-sum assumption simplifies equilibrium
Information	Complete vs. partial	Defender only estimates the attacker state

2.9.3 Attacker-Defender Games

Once malicious traffic is detected, the defender must decide how to respond in a way that balances protection, service continuity, and operational cost. A payoff matrix expresses the utility of each defensive action under each attacker strategy, and the defender selects the action that maximizes its expected utility. A single, static game captures only one instant; an adaptive attacker over many rounds requires the Markov chains and stochastic games described next.

2.10 Markov Chains and Stochastic Games

2.10.1 Markov Chains

A Markov chain describes a system moving among a finite set of states, where the probability of the next state depends only on the current state (the Markov property):

$$P(X_{t+1} = j \mid X_t = i, \ldots, X_0) = P(X_{t+1} = j \mid X_t = i)$$

The chain is described by a transition matrix P, with each row summing to one:

$$\Sigma_j \; P(i, j) = 1, \quad \text{for every state } i$$

In this project, the attacker is modeled as a Markov chain over four states: S0 (idle), S1 (slow stealth flooding), S2 (intensive flooding), and S3 (IP spoofing). Its long-term behavior is given by the stationary distribution π:

$$\pi P = \pi, \quad \text{with} \quad \Sigma_i \; \pi(i) = 1$$

This lets the project represent a time-varying attacker reproducibly and design the slow stealth state to remain below conventional detection thresholds.

2.10.2 From Markov Decision Processes to Stochastic Games

A Markov Decision Process extends a Markov chain with actions and rewards for a single decision-maker. A stochastic (Markov) game generalizes it to two or more players interacting across states, where the state evolves according to the players' joint actions. Here the defender is

the Ryu controller, with actions ALLOW, RL_1, RL_2, RL_3, and BLOCK, and the attacker is the Markov-modeled source. The defender seeks a policy maximizing the expected discounted reward:

$$V(s) = max_a \ \Sigma_\{s'\} \ [\, R(s, a) + \beta \cdot P(s' \mid s, a) \cdot V(s') \,]$$

where R(s, a) is the immediate reward, β is a discount factor between zero and one, and V(s) is the value of state s.

2.10.3 Solving and Implementing the Game

Because the state and action sets are small, the equilibrium policy can be computed either by assuming a zero-sum game and solving a small linear program per state (Shapley value iteration) or by learning the policy through Minimax-Q learning. The optimal policy is computed offline and stored as a lookup table, keeping the runtime decision fast and the controller lightweight, with a switching cost to prevent rapid oscillation. Because the defender only estimates the attacker's state, the interaction is formally a partially observed stochastic game; the project adopts the accepted simplification of using the most likely estimated state as the game state, and states this assumption explicitly.

2.11 Chapter Summary

This chapter presented the theoretical foundations of the project, covering Software-Defined Networking and its layered architecture, OpenFlow, the Ryu controller and traffic mirroring in Open vSwitch, intrusion detection systems and Snort, DDoS attacks and IP spoofing in SDN, and the advanced SDN-specific attacks targeted for extension. It then introduced game theory, Markov chains, Markov decision processes, and stochastic games as the mathematical basis for representing an intelligent, adaptive attacker and computing an optimal, time-aware defensive policy, preparing the ground for the design presented in the following chapters.

Conclusion

This report established the motivation, problem, and theoretical foundations for an adaptive defense framework that protects SDN networks against intelligent, Markovian stealth attacks using stochastic game theory. The introduction explained why SDN security remains critical, defined the problem of the controller as a single point of failure against adaptive attackers, stated the objective of combining Markov chains and stochastic games, reviewed ten recent (2025–2026) studies, and identified the project's practical applications and scientific challenges.

The theoretical chapter then provided the foundations needed for the design: Software-Defined Networking and its architecture, OpenFlow, the Ryu controller, Open vSwitch mirroring, intrusion detection and Snort, DDoS attacks and IP spoofing, advanced SDN-specific attacks, and the mathematics of game theory, Markov chains, and stochastic games. Together, these chapters prepare the ground for the next stage of the project: designing and implementing the adaptive decision engine, extending the threat vectors, and evaluating the framework with statistical validation of the experimental results.

References

Base reference study: F. Razvan and C. Mitica, "Enhancing network security through integration of game theory in a software-defined networking framework," Int. J. Inf. Secur., vol. 24, art. 100, 2025, doi: 10.1007/s10207-025-01012-4.

[1] W. Shao, Y. Wang, R. He, M. E. Ahmed, and S. Camtepe, "AdaDoS: Adaptive DoS attack via deep adversarial reinforcement learning in SDN," arXiv:2510.20566, 2025.

[2] A. Kachavimath and D. G. Narayan, "Hybrid approach for detection and mitigation of DDoS attacks using multi-feature selection, unsupervised learning, and game theory," J. Telecommun. Inf. Technol., no. 4, 2025, doi: 10.26636/jtit.2025.4.2261.

[3] R. McGugan, M. M. Cook, and D. Pezaros, "Multi-agent reinforcement learning for DoS mitigation in industrial control systems," Comput. Netw., vol. 288, art. 112668, 2026, doi: 10.1016/j.comnet.2026.112668.

[4] Z. You, L. Mai, S. Suo, Q. Lin, and Z. Xiong, "In-depth analysis of DDoS anomaly detection in software-defined networks," J. Wireless Commun. Netw., vol. 2026, art. 116, 2026, doi: 10.1186/s13638-026-02630-9.

[5] M. N. Swileh and S. Zhang, "Proactive DDoS detection and mitigation in decentralized SDN via port-level monitoring and zero-training large language models," arXiv:2511.00460, 2025.

[6] D. Tang, P. Tan, Y. Yan, K. Li, W. Liang, Z. Qin, and J. Zhang, "Low-rate flow table overflow attacks in SDN: Model, analysis and a machine learning based mitigation approach," IEEE Trans. Comput., vol. 75, no. 6, pp. 2253-2266, 2026, doi: 10.1109/TC.2026.3677483.

[7] Z. Ji, Y. Cui, Y. Guo, G. Shen, Y. Chen, and C. Guo, "Towards saturation attack detection in SDN: A multi-edge representation learning-based method," J. King Saud Univ. Comput. Inf. Sci., vol. 37, no. 6, pp. 1-22, 2025, doi: 10.1007/s44443-025-00149-5.

[8] B. Irtaish and M. Hamarsheh, "Enhanced real-time DDoS detection and mitigation in SDN using ONOS controller," in Proc. Engineering for Palestine Conf. (ENG4PAL), Palestine Polytechnic Univ., 2025.

[9] I. A. Mahar, K. Aziz, P. Chakrabarti, N. Ahmed, M. Ladan, and Y. Javed, "A hybrid machine learning approach for detecting DDoS attacks in software-defined networks," Sci. Rep., vol. 16, art. 6533, 2026, doi: 10.1038/s41598-026-35458-w.

[10] C. L. Kumar, S. Betam, D. Pustokhin, E. L. Lydia, K. Bala, R. Aluvalu, and B. S. Panigrahi, "Metaparameter optimized hybrid deep learning model for next generation cybersecurity in SDN environment," Sci. Rep., vol. 15, 2025, doi: 10.1038/s41598-025-96153-w.